

Presentation speech

Speaker: Afnan · about 14 minutes in full, 11 with the ∞ lines skipped (at ~145 words a minute) · bold = numbers to say clearly · ► brackets = what to do, not what to say · ∞ = you may skip from this mark to the end of the paragraph

Every number here is taken from the thesis report as submitted. Where the slide wording and the report disagree, the speech follows the report, so what you say is always defensible.

SLIDE 1 Title

1:02 (cut: 0:57)

Good morning, respected members of the panel.

We are group P25301051. I am Afnan Mazumdar, and with me are Jerin Aktar and Shoyeb Hasan Sayem. Our supervisor is Utsho Kumar Roy sir.

Our thesis is titled *AI-Powered Legal Document Analysis System: Clause Detection, Category-Level Risk Prioritization and Plain-Language Explanation on CUAD*.

In simple words, we built a **three-model pipeline** that reads a commercial contract, finds which important clauses it contains, extracts the clause text, gives each clause a risk level, and adds a plain-English sentence. ∞ Every number we report comes from contracts the models never saw during training.

I want to say one thing at the start. **Three of our five research questions came back with the answer "no."** We are not hiding those results. Each one is a measurement that someone looking only at the individual component scores would have got wrong, and we present them as our contribution.

► click

SLIDE 2 The problem, and the data

1:54 (cut: 1:37)

Let me start with the problem. Contracts are long, and the clauses that matter are short.

In our dataset, the median contract is about **33,000 characters**, roughly five thousand words. The median clause a lawyer actually highlighted is only **196 characters**. A standard transformer reads 512 tokens at a time, which is about 2,000 characters. So a typical contract is about **sixteen times** longer than what the model can read in one pass.

The dataset is CUAD, the Contract Understanding Atticus Dataset. It has **510 real commercial contracts** from public SEC filings, and lawyers marked every clause belonging to **41 categories**: governing law, non-compete, liability caps, IP assignment, and so on.

► point to the right-hand cards

We split the data **80/20 at the contract level**, with a fixed seed of 42: **408 contracts for training and 102 for testing**. Whole contracts go to one side or the other, so no clause from a training contract can reach the test set. ∞ All three models use this same split, so "test" always means the same 102 unseen contracts.

∞ ► point to the chart This chart compares the positive rate of every category in the training and test halves. They track each other closely, with a correlation of 0.986.

The data is also severely imbalanced. Document Name appears in 100 percent of contracts, while Source Code Escrow appears in only 2.5 percent. **Five categories have seven or fewer positive contracts in our test set, and they score zero F1.**

And the dataset cannot be grown. Its annotation reportedly cost about **two million dollars** of attorney time. So transfer learning from pre-trained models is mandatory, overfitting is the main enemy, and every result we report is on held-out contracts.

► click

SLIDE 3 The pipeline, and how it reads a contract

2:46 (cut: 2:24)

This is our pipeline. It has four stages.

Stage one, presence. For each of the 41 categories, decide whether the contract contains that clause. We use two models here: a full-document TF-IDF baseline and a windowed DistilBERT.

Stage two, span. For each category we find, extract the exact clause text, using DistilBERT trained for question answering.

Stage three, risk. Attach a High, Medium or Low level to that category. We wrote these levels ourselves, from the side of the weaker party. ∞ We used five criteria: financial exposure, irreversibility, restriction of future freedom, asymmetry, and duration. That gives 8 High, 22 Medium and 11 Low categories.

Stage four, explanation. FLAN-T5 adds a one-line plain-English sentence.

► point to the red and green boxes These boxes show what each boundary costs, measured end to end on the test set. Presence loses **22 percent** of the gold clauses. Span loses another **56.3 percent** of the original total. Risk loses nothing, because it is a lookup: it is right whenever the category is right. And the explanation picks the wrong template **30 percent** of the time.

► point to the window diagram Now, how does the model read a long contract? We slide a **2,000-character window** across the text, moving **1,500 characters** each step. That leaves a **500-character overlap**, which is longer than the median clause of 196 characters, so a typical clause falls completely inside at least one window. ∞ A median contract gives **22 windows**, and the longest gives **226**.

The model scores every window, and the contract's score is the **maximum**. If any window fires, as window four does here at 0.91, the clause is reported present. This is the multiple-instance learning rule. ∞ The contract is a bag of windows, and the bag is positive if any window is positive.

This rule has one benefit and one cost. The benefit: **it removes the recall ceiling**, because every part of the contract falls inside some window. The cost is **precision**: one wrong window anywhere is enough to flip the whole contract to positive.

We chose this design because of a measurement. Before training anything, we tested the common alternative, retrieval, which sends only the top few windows to the model. Even in its best setting, retrieval found the right window only **64 percent** of the time. That is a hard cap on recall, already below the baseline's recall of 0.81. So we dropped retrieval after a check that took seconds, instead of wasting a training run.

► click

SLIDE 4 How we trained, and how we test

2:07 (cut: 1:32)

All three models trained on one laptop: an **Apple M4 with 16 gigabytes** of memory, on Apple's Metal backend, with no CUDA GPU.

► point to the three rows The **presence model** is DistilBERT, trained on **24,000 of the 53,662** available windows, with batch size 16 and learning rate 2e-5. It took about **49 minutes**. The **span model** is DistilBERT for question answering, trained on 8,000 of 11,156 examples with 320-token inputs, in about **23 minutes**. The **explanation model** is FLAN-T5-small, trained on 4,000 pairs on the CPU, in about **8 minutes**.

⌘ The plots show the learning-rate schedules: AdamW, linear decay, no warmup, and gradient clipping at 1.0.

⌘ Two engineering notes. We first tried DeBERTa-v3, but it produced NaN gradients on the Mac, so we switched to DistilBERT. And in our full-data run, gradient clipping was active on **98.4 percent** of steps, so clipping, not only the small learning rate, is what kept training stable.

► point to the right-hand cards What did we hold out? For presence, **5 percent of the training windows**, used only for monitoring. For span and the summarizer, nothing. And **no hyperparameter was ever tuned**. Every value is a standard default. That fully protects the test set, but it also means the transformer competed without any tuning.

We also made **one mistake**, and we report it. An early exploratory run of the summarizer monitored its loss on 300 clauses from the **test** split. That run saved no checkpoint, and the summarizer we report was rebuilt with no evaluation set at all, so no reported result depends on it. But we should not have done it.

For testing: **102 unseen contracts, 4,182 presence decisions, and 2,667 span windows**. For confidence intervals we use a **cluster bootstrap**. We resample **whole contracts, not individual cells**. ⌘ Each contract produces 41 decisions that move together, so contracts are the right unit, and we resample them ten thousand times.

► click

SLIDE 5 RQ1: the transformer loses to a bag of words

1:37 (cut: 1:19)

Now our first research question. **Can a sliding-window transformer beat a full-document lexical baseline at deciding which clauses are present? The answer is no.**

► point to the bar chart This is micro-F1 over all 4,182 test decisions. The **TF-IDF baseline scores 0.775**. The **windowed transformer alone scores 0.694**. The bag of words wins by **0.081**. In the bootstrap, the baseline wins in all ten thousand resamples, with a confidence interval of 0.063 to 0.099. That is the finding we stand behind.

⌘ Why does it happen? Legal drafting is very formulaic, so vocabulary alone carries a lot of signal, and TF-IDF reads the entire contract. The transformer reads windows, and taking the maximum over many windows creates false positives. Its precision is only **0.56**.

The **AND ensemble**, which reports a clause only when both models agree, scores **0.776** and is nominally first. But we **withdrew** that claim. Its lead is **0.0014**, about one decision in a thousand. The bootstrap interval runs from minus 0.007 to plus 0.009, so it crosses zero, and the ensemble wins only 63 percent of resamples. The top two bars are a tie.

► point to the small chart This shows per-category F1. Frequent, formulaic categories are almost solved: **Document Name 1.000, Parties 0.995, Governing Law 0.961**. But the five categories with seven or fewer test positives score **zero**. That is why the baseline's macro-F1 is only **0.572** while its micro-F1 is 0.775, and why we use micro-F1 as our headline.

► click

SLIDE 6 We ranked a leaderboard by noise

2:07 (cut: 1:56)

Before accepting that the transformer simply loses, we tested every explanation we could, one at a time.

► point to the bullets as you go

Was it undertrained? It saw only 24,000 of 53,662 windows, so we retrained it on all of them. The score changed by **minus 0.0004**. No effect.

Was it an unlucky seed? We trained with three seeds. The gap to the baseline is **9.5 standard deviations** of the seed variation. ∞ Even the best seed still loses by 0.067.

Was max-pooling the wrong rule? We tried **six aggregation rules** on the same window scores. None of them reaches the baseline, even at its own best threshold.

Was the threshold wrong? This one is a real caveat. The 0.5 threshold was never chosen; it is just the default. Sweeping it recovers **0.056**, but we found that on the test set, so it is an upper bound, and it still does not reach the baseline. I will come back to it on the next slide.

So our conclusion: **the transformer loses. It is not a budget, seed or aggregation artefact.**

► point to the chart This chart shows why we withdrew the ensemble's lead. The red bar is the lead itself: **0.0014**. Retraining with different seeds moves the ensemble by **0.0022**. Redrawing the train-test split moves the baseline by **0.0063**; we refitted it on 20 different splits. And the bootstrap half-width is **0.0078**. **Every source of uncertainty is larger than the result.** Split variance is the largest, and reseeding cannot see it.

≈ Our own split actually scored below the 20-split average, so it was not a lucky split for the baseline.

So why keep the ensemble at all? Removing it changes nothing measurably, which is itself a finding. But it has two real benefits: **balanced precision and recall, 77 and 78 percent**, and it is **four times less sensitive to the training seed** than the transformer inside it.

► click

SLIDE 7 The threshold, and what the confidence means

2:13 (cut: 1:44)

Two more things we only measured after the fact, and both turned out to matter: the threshold, and what the confidence score means.

► point to the left plot **First, the threshold.** Every decision compares a score to **0.5**, which is simply the softmax default. This plot sweeps it. For the transformer, the best point is 0.9, where micro-F1 rises from **0.694 to 0.750**. The reason is structural: the maximum over about 35 windows is pushed towards one, so a cut at 0.5 lets in too much.

But **we do not adopt 0.9.** We found it by looking at the test data, and choosing a threshold on test data is exactly the tuning our protocol forbids. ≈ Also, for the AND ensemble, 0.5 is already the best point, and for the transformer, moving to 0.9 lowers recall on High-risk clauses from 0.84 to 0.69. No single threshold is best for both the average and the clauses that matter most.

► point to the right plot **Second, calibration.** This is a reliability diagram. Our transformer's **Expected Calibration Error is 0.201**. When it scores above 0.9, the clause is actually present only about **73 percent** of the time. Between 0.5 and 0.6, it is present **less than 20 percent** of the time. Our app shows this score as a confidence bar, so a full bar really means about 73 percent, not 100.

► point to the bottom line The model also over-fires by design. Precision at 0.5 is **0.562**. ✂
When we gave it more training data, recall went up from **0.908 to 0.927**, but precision went down. A maximum over 35 windows is 35 chances to be wrong.

► point to the green card What would we do about it? **Carve a validation split from the 408 training contracts**, choose the threshold there, and fit a calibration map such as Platt scaling on the same split. That fixes both problems without retraining and without touching the test set.

That completes the presence stage. Now **[teammate's name]** will show what these errors mean for the clauses a signer cares about most.

► hand over

Timing

At about 145 words a minute.

SLIDE	FULL	CUT VERSION (SKIP ×)
1 Title	1:02	0:57
2 Problem & data	1:54	1:37
3 Pipeline	2:46	2:24
4 Training & testing	2:07	1:32
5 RQ1	1:37	1:19
6 Leaderboard by noise	2:07	1:56
7 Threshold & confidence	2:13	1:44
Total	13:46	11:30

If your slot for slides 1–7 is shorter than the cut version, slide 4's model rows and slide 6's chart paragraph are the next safest places to shorten.

Three sentences the speech deliberately says differently from the slides

- **Slide 1:** say "Utsho Kumar Roy" (the slide spells it "Utsha").
- **Slide 5:** say "the **baseline's** macro-F1 is 0.572". The slide places 0.572 next to the ensemble's 0.776, but 0.572 belongs to the TF-IDF baseline (report §5.3).
- **Slide 7:** say "about 35 windows". The slide prints "-35".

Backup answer: "Does the presence model read the whole 2,000-character window?"

We checked the code. Training and the app both tokenize the (question, window) pair with `truncation=True, max_length=256` and no stride. The CUAD questions take about 53 tokens at the median, so the model reads roughly the **first 1,000 characters** of each 2,000-character window. That is about 700 characters for the longest question and 1,100 for the shortest, measured on the two sample contracts in the app folder.

If asked, answer plainly:

"No. At 256 tokens the presence model reads about the first half of each window. That isn't discussed in the report. It's a likely extra reason the transformer loses to TF-IDF, which reads the whole contract, and it may add to the wrong-window problem in the span stage. None of our four controls changes the windowing, so they can't rule it out. The fix is cheap: make the windows match the token limit, around 900 characters, with a stride that keeps the overlap longer than the median clause. The finding still holds for the system we built; this is one more explanation for it."

Only give this answer if someone asks. Don't raise it in the speech.